

Conversiono AI Platform

Epic 5: Merchant Dashboard v1

Comprehensive Course Documentation

Jira Stories: CONV-51 through CONV-55

Next.js 14 | React 18 | TypeScript 5 | Tailwind v4 | Recharts

Sprint 5 - March 2026

16 Chapters | 7 Dashboard Pages | 26 API Endpoints | Full Component Library

Table of Contents

- 1 Architecture Overview
- 2 Technology Stack & Project Setup
- 3 API Client Layer (CONV-51)
- 4 Custom React Hooks
- 5 Dashboard Layout & Navigation (CONV-51)
- 6 Reusable UI Component Library
- 7 Overview Dashboard Page
- 8 Sessions List & Filtering (CONV-52)
- 9 Session Detail View (CONV-53)
- 10 Analytics Dashboard (CONV-54)
- 11 Experiments Management (CONV-55)
- 12 Interventions Dashboard (CONV-55)
- 13 Settings & SDK Key Management
- 14 Formatting Utilities
- 15 Performance Optimization Deep Dive
- 16 Build, Deploy & Next Steps

Chapter 1

Architecture Overview

1.1 Layered Architecture

The Conversiono frontend follows a clean layered architecture that separates concerns into distinct tiers. Each layer has a single responsibility and communicates only with its adjacent layers. This pattern ensures maintainability, testability, and scalability as the application grows.

At the highest level, the architecture consists of four layers:

1. **Presentation Layer (Pages & Components):** React components that render UI and handle user interactions. Each page lives under `app/` using Next.js App Router conventions.
2. **State & Data Layer (Hooks):** Custom hooks like `useQuery` that manage server state, caching, and data fetching lifecycle. This layer abstracts away the complexity of API communication.
3. **API Client Layer (`lib/api.ts`):** A typed fetch wrapper that handles HTTP communication, timeouts, authentication headers, and error normalization.
4. **External Services:** The backend REST API, which provides sessions, analytics, experiments, interventions, and merchant management endpoints.

Why Layered Architecture?

By isolating each concern, you can swap implementations without affecting other layers. For example, you could replace the fetch-based API client with Axios or GraphQL without changing any page component.

1.2 Directory Structure

The project follows Next.js 14 App Router conventions where each folder under `app/` represents a URL route. Components, hooks, and utilities live in dedicated directories outside the `app/` folder for reusability.

```
frontend/src/
  app/
    layout.tsx          # Root layout (html + body)
    page.tsx            # Redirect to /dashboard
    globals.css         # Tailwind v4 + design tokens
  dashboard/
    layout.tsx          # Dashboard shell wrapper
    page.tsx            # Overview (KPIs, funnel, risk)
  sessions/
    page.tsx            # Session list with filters
    [id]/page.tsx       # Session detail view
  analytics/page.tsx    # Full analytics dashboard
  experiments/page.tsx  # Experiment CRUD + stats
  interventions/page.tsx # Intervention performance
  settings/page.tsx     # SDK key + merchant profile
  components/
    charts/LazyRecharts.tsx # Lazy-loaded chart wrappers
    layout/Sidebar.tsx      # Navigation sidebar
    layout/DashboardShell.tsx # Shell with responsive sidebar
    ui/                     # Card, Badge, Spinner, etc.
  lib/
    api.ts              # Typed API client
    hooks.ts            # useQuery with caching
    format.ts           # Formatting utilities
```

1.3 Data Flow

Data flows unidirectionally through the application. Understanding this flow is critical for debugging and extending the dashboard:

1. User navigates to a page (e.g., /dashboard/sessions).
2. The page component calls useQuery with an API endpoint function (e.g., getSessions).
3. useQuery checks its in-memory cache. If a valid entry exists (< 30s old), it returns cached data immediately.
4. If cache misses, useQuery calls the API client function, which issues a fetch() with AbortController timeout.
5. The API client attaches the X-SDK-Key header from localStorage and sends the request to the backend.
6. The response is parsed, cached, and returned to the component, which renders the UI.
7. If the user navigates away before the response arrives, the cancelled flag prevents stale state updates.

Important: Server Components vs Client Components

All dashboard pages use 'use client' because they depend on browser APIs (localStorage, useState, useEffect). The root layout.tsx is a Server Component that wraps the client tree.

1.4 Jira Story Mapping

Epic 5 consists of five Jira stories, each delivering a vertical slice of functionality:

Story	Title	Scope
CONV-51	Dashboard Foundation	Layout, API client, hooks, overview
CONV-52	Session Explorer	Session list, filters, pagination
CONV-53	Session Detail	Detail view, features, timeline
CONV-54	Analytics Dashboard	Charts, cohorts, friction analysis
CONV-55	Experiments & More	CRUD, interventions, settings

Chapter 2

Technology Stack & Project Setup

2.1 Core Dependencies

The project is built on a modern React stack optimized for developer experience and runtime performance. Every dependency was chosen deliberately to minimize bundle size while maximizing capability.

Package	Version	Purpose
Next.js	14.x	App Router, SSR, file-based routing, image optimization
React	18.x	Component model, hooks, concurrent features
TypeScript	5.x	Static typing with strict mode for safety
Tailwind CSS	v4	Utility-first CSS with HSL design tokens
Recharts	3.8	Composable chart library built on D3
clsx	latest	Conditional className merging utility

Why No State Management Library?
We deliberately avoided Redux, Zustand, or Jotai. The useQuery hook with its built-in cache handles all server state. Local UI state (filters, form inputs) is managed with useState. This keeps the dependency tree lean and the mental model simple.

2.2 Tailwind v4 and HSL Design Tokens

Tailwind CSS v4 introduces a new CSS-first configuration approach. Instead of a tailwind.config.js file, design tokens are defined as CSS custom properties in globals.css. This is a major shift from Tailwind v3.

The design system uses HSL (Hue, Saturation, Lightness) color values stored without the hsl() wrapper. This allows dynamic opacity via the syntax hsl(var(--primary)/0.5).

```
/* globals.css - Design Token Excerpt */
:root {
  --background: 0 0% 100%;
  --foreground: 222.2 84% 4.9%;
  --primary: 221.2 83.2% 53.3%;
  --primary-foreground: 210 40% 98%;
  --secondary: 210 40% 96.1%;
  --muted: 210 40% 96.1%;
  --muted-foreground: 215.4 16.3% 46.9%;
  --destructive: 0 84.2% 60.2%;
  --border: 214.3 31.8% 91.4%;
  --ring: 221.2 83.2% 53.3%;
  --radius: 0.5rem;
}
```

Using HSL Tokens in Components

To use these tokens in Tailwind classes, you reference them with the `hsl()` function and `var()` syntax. The key insight is that the CSS variable holds the raw HSL values without the `hsl()` wrapper, enabling opacity modifiers.

```
// Usage in a component
<div className="bg-[hsl(var(--primary))]">
  Solid primary background
</div>
<div className="bg-[hsl(var(--primary)/0.5)]">
  50% opacity primary background
</div>
<div className="text-[hsl(var(--muted-foreground))]">
  Muted text color
</div>
```

2.3 TypeScript Strict Mode

The project uses TypeScript strict mode, which enables all strict type-checking options including `strictNullChecks`, `noImplicitAny`, and `strictFunctionTypes`. This catches entire classes of bugs at compile time rather than runtime.

Every API response is typed with explicit interfaces. The API client returns typed promises, and `useQuery` is generic over the response type. This creates a fully typed pipeline from the API to the rendered component.

```
// Typed API response example
interface Session {
  session_id: string;
  visitor_id: string;
  outcome: 'converted' | 'abandoned' | 'bounced';
  risk_score: number;
  device_type: string;
  created_at: string;
}

// useQuery infers the type from the fetcher
const { data, loading, error } = useQuery<Session[]>(
  'sessions',
  () => getSessions(filters)
);
```

2.4 Project Setup Steps

1. Clone the repository and navigate to the frontend/ directory.
2. Run 'npm install' to install all dependencies from package.json.
3. Copy .env.example to .env.local and configure NEXT_PUBLIC_API_URL (defaults to http://localhost:8000).
4. Run 'npm run dev' to start the Next.js development server on port 3000.
5. Ensure the backend API is running on port 8000 for data to load.
6. Navigate to http://localhost:3000 which redirects to /dashboard automatically.

CORS Configuration Required

The backend must allow requests from localhost:3000 (and 3001 for alternate dev ports). The backend config.py was updated to include both origins. Verify CORS_ORIGINS in backend .env if you see errors.

Chapter 3

API Client Layer (CONV-51)

3.1 The request() Function

The core of the API layer is a single `request()` function in `src/lib/api.ts`. This function wraps the native `fetch()` API with three critical features: automatic timeout via `AbortController`, authentication header injection, and error normalization. Every API call in the application flows through this function.

```
const API = process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:8000';

class ApiError extends Error {
  constructor(
    public status: number,
    public detail: string
  ) {
    super(detail);
    this.name = 'ApiError';
  }
}

async function request<T>({
  path: string,
  options: RequestInit = {}
}): Promise<T> {
  const ctrl = new AbortController();
  const timer = setTimeout(() => ctrl.abort(), 15_000);

  const sdkKey =
    typeof window !== 'undefined'
      ? localStorage.getItem('sdkKey') ?? ''
      : '';

  try {
    const res = await fetch(`${API}${path}`, {
      ...options,
      signal: ctrl.signal,
      headers: {
        'Content-Type': 'application/json',
        'X-SDK-Key': sdkKey,
        ...options.headers,
      },
    });
    if (!res.ok) {
      const body = await res.json().catch(() => ({}));
      throw new ApiError(res.status, body.detail ?? res.statusText);
    }
    return (await res.json()) as T;
  } catch (err) {
    if (err instanceof DOMException && err.name === 'AbortError') {
      throw new ApiError(0, 'Request timed out after 15s');
    }
    throw err;
  } finally {
    clearTimeout(timer);
  }
}
```

AbortController Pattern

AbortController is the modern way to cancel fetch requests. By creating one per request and aborting after 15 seconds, we prevent hanging requests from blocking the UI. The finally block ensures the timer is always cleared, preventing memory leaks.

3.2 Authentication Pattern

Authentication uses a simple SDK key stored in localStorage. The key is read on every request and sent as the X-SDK-Key header. This pattern was chosen for its simplicity in a merchant dashboard context where the SDK key identifies the merchant account.

The `typeof window !== 'undefined'` guard is essential in Next.js because code can execute on the server during SSR where localStorage is not available. This guard prevents ReferenceError crashes during server-side rendering.

Security Note: localStorage Limitations

localStorage is vulnerable to XSS attacks. For production, consider httpOnly cookies or a backend-for-frontend (BFF) token exchange. The current pattern is acceptable for an internal merchant dashboard but should be hardened before public deployment.

3.3 Custom ApiError Class

The ApiError class extends the native Error with two additional properties: status (HTTP status code) and detail (human-readable message). This enables downstream code to handle errors based on status codes (401 for auth, 404 for not found, etc.).

When the response is not OK, the function attempts to parse the JSON body for a detail field (following FastAPI's error convention). If parsing fails, it falls back to the HTTP status text.

3.4 Endpoint Coverage

The API client provides 26 typed functions across 6 categories. Each function calls `request<T>()` with the appropriate path and HTTP method.

Category	Function	Method	Path
Sessions	<code>getSessions()</code>	GET	<code>/api/v1/sessions</code>
Sessions	<code>getSession(id)</code>	GET	<code>/api/v1/sessions/:id</code>
Sessions	<code>getSessionEvents(id)</code>	GET	<code>/api/v1/sessions/:id/events</code>
Analytics	<code>getConversionFunnel()</code>	GET	<code>/api/v1/analytics/funnel</code>
Analytics	<code>getFrictionPoints()</code>	GET	<code>/api/v1/analytics/friction</code>
Analytics	<code>getRiskDistribution()</code>	GET	<code>/api/v1/analytics/risk</code>
Analytics	<code>getCohortAnalysis()</code>	GET	<code>/api/v1/analytics/cohorts</code>

Experiments	getExperiments()	GET	/api/v1/experiments
Experiments	createExperiment()	POST	/api/v1/experiments
Experiments	getExperimentStats()	GET	/api/v1/experiments/:id/stats
Interventions	getInterventions()	GET	/api/v1/interventions
Interventions	getInterventionStats()	GET	/api/v1/interventions/stats
Merchant	getMerchant()	GET	/api/v1/merchant
Merchant	updateMerchant()	PUT	/api/v1/merchant

Type Safety End-to-End

Each endpoint function specifies its return type via the generic parameter: `request<Session[]>('/api/v1/sessions')`. TypeScript then enforces correct usage in every component that consumes the data. A type mismatch is caught at compile time, not in production.

Chapter 4

Custom React Hooks

4.1 The useQuery Hook

The `useQuery` hook is the backbone of all data fetching in the application. It provides an SWR-like (stale-while-revalidate) pattern without any external dependencies. The hook manages loading states, error handling, caching, and automatic cancellation.

```
interface QueryResult<T> {  
  data: T | null;  
  loading: boolean;  
  error: string | null;  
  refetch: () => void;  
}  
  
function useQuery<T>(  
  key: string,  
  fetcher: () => Promise<T>,  
  deps: unknown[] = []  
): QueryResult<T> { ... }
```

The hook accepts three parameters: a cache key string, a fetcher function that returns a promise, and an optional dependency array. It returns an object with data, loading, error, and a refetch function.

4.2 In-Memory Cache System

The cache is a module-level Map that persists across renders but not across page refreshes. Each entry stores the data and a timestamp. Entries older than 30 seconds are considered stale and trigger a re-fetch.

```
const CACHE_TTL = 30_000; // 30 seconds
const MAX_CACHE = 100;    // Maximum entries

interface CacheEntry<T> {
  data: T;
  ts: number;
}

const cache = new Map<string, CacheEntry<unknown>>();

function pruneCache() {
  if (cache.size > MAX_CACHE) {
    // Delete oldest entries (LRU approximation)
    const oldest = [...cache.entries()]
      .sort((a, b) => a[1].ts - b[1].ts)
      .slice(0, cache.size - MAX_CACHE);
    oldest.forEach(([k]) => cache.delete(k));
  }
}
```

Why Module-Level Cache?

By defining the cache outside any component, it becomes a singleton shared across all `useQuery` instances. This means navigating between pages reuses cached data, providing instant page transitions for recently visited routes. The 30s TTL prevents stale data.

4.3 The `fetcherRef` Pattern

A subtle but critical pattern in `useQuery` is the `fetcherRef`. The `fetcher` function often captures state variables in its closure (like filter values). If we put the `fetcher` directly in the `useEffect` dependency array, it would cause infinite re-renders because functions are recreated on every render.

```
const fetcherRef = useRef(fetcher);
fetcherRef.current = fetcher;

useEffect(() => {
  let cancelled = false;

  // Use fetcherRef.current so the effect doesn't
  // depend on the fetcher identity
  fetcherRef.current().then(data => {
    if (!cancelled) {
      setData(data);
      cache.set(key, { data, ts: Date.now() });
      pruneCache();
    }
  });

  return () => { cancelled = true; };
}, [key, depsKey, tick]);
```

Common Pitfall: Stale Closures

Without `fetcherRef`, the `useEffect` would close over an old version of the `fetcher` function. When the effect fires, it would use stale filter values. The ref always points to the latest function, solving this classic React hooks gotcha.

4.4 Dependency Tracking via `JSON.stringify`

The `deps` array is serialized to a string using `JSON.stringify`, creating a stable dependency key. This allows `useQuery` to detect when filter parameters change without relying on object reference equality.

```
// Inside useQuery:
const depsKey = JSON.stringify(deps);

// Used in useEffect dependency array:
useEffect(() => { ... }, [key, depsKey, tick]);

// Usage in a component:
const { data } = useQuery(
  'sessions',
  () => getSessions(filters),
  [filters.outcome, filters.device, filters.page]
);
```

4.5 Cancellation on Unmount

The `'let cancelled = false'` pattern prevents state updates after a component unmounts. When the user navigates away, the cleanup function sets `cancelled = true`. Any pending API response will be ignored, preventing React's 'setState on unmounted component' warning.

4.6 Manual Refetch

The `refetch` function increments a tick counter, which is in the `useEffect` dependency array. This triggers a re-fetch without changing the cache key or dependencies.

```
const [tick, setTick] = useState(0);
const refetch = useCallback(() => setTick(t => t + 1), []);

// Returned to the caller:
return { data, loading, error, refetch };
```


Chapter 5

Dashboard Layout & Navigation (CONV-51)

5.1 DashboardShell Component

The DashboardShell is the outermost layout component for all dashboard pages. It provides a responsive sidebar and main content area. The shell is used in the dashboard/layout.tsx file, which wraps all child routes.

```
// dashboard/layout.tsx
import DashboardShell from '@components/layout/DashboardShell';

export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return <DashboardShell>{children}</DashboardShell>;
}
```

The DashboardShell renders a flex container with the Sidebar on the left and the children content area on the right. On mobile viewports, the sidebar collapses into a hamburger menu.

```
// DashboardShell.tsx (simplified)
export default function DashboardShell({
  children
}: { children: React.ReactNode }) {
  const [open, setOpen] = useState(false);

  return (
    <div className="flex h-screen bg-[hsl(var(--background))]">
      <Sidebar open={open} onClose={() => setOpen(false)} />
      <div className="flex-1 overflow-auto">
        <header className="md:hidden p-4">
          <button onClick={() => setOpen(true)}>Menu</button>
        </header>
        <main className="p-6">{children}</main>
      </div>
    </div>
  );
}
```

5.2 Sidebar Navigation

The Sidebar component renders the Conversiono logo, navigation links, and handles active state highlighting. It uses Next.js `usePathname()` to determine which link is currently active.

```
// Sidebar.tsx navigation items
const NAV_ITEMS = [
  { href: '/dashboard', label: 'Overview', icon: HomeIcon },
  { href: '/dashboard/sessions', label: 'Sessions', icon: UsersIcon },
  { href: '/dashboard/analytics', label: 'Analytics', icon: ChartIcon },
  { href: '/dashboard/experiments', label: 'Experiments', icon: FlaskIcon },
  { href: '/dashboard/interventions', label: 'Interventions' },
  { href: '/dashboard/settings', label: 'Settings', icon: GearIcon },
];
```

Each navigation item is a Next.js Link component with conditional styling. The active state uses `bg-[hsl(var(--primary)/0.1)]` for a subtle highlight and `text-[hsl(var(--primary))]` for the text color.

Next.js Image priority Prop

The Sidebar logo uses `<Image priority />` to tell Next.js to preload this image. Since the logo is above the fold on every page, it impacts Largest Contentful Paint (LCP). The priority prop adds a `<link rel='preload'>` to the document head.

5.3 Responsive Design Strategy

The dashboard uses Tailwind's responsive prefixes (`md:`, `lg:`) to adapt the layout. The sidebar is hidden on mobile (`hidden md:flex`) and the hamburger menu appears only on small screens (`md:hidden`). This approach avoids JavaScript-based responsive logic in favor of CSS media queries.

1. Mobile (< 768px): Sidebar hidden, hamburger button visible, full-width content.
2. Tablet (768px+): Sidebar visible as narrow strip with icons only.
3. Desktop (1024px+): Full sidebar with labels, spacious content area.

Chapter 6

Reusable UI Component Library

6.1 Card & CardHeader / CardBody

The Card component is the fundamental container used throughout the dashboard. It provides consistent rounded borders, shadows, and padding. CardHeader and CardBody subcomponents handle internal spacing.

```
// Card.tsx
export function Card({
  children, className = ''
}: { children: React.ReactNode; className?: string }) {
  return (
    <div className={clsx(
      'rounded-2xl border border-[hsl(var(--border))]',
      'bg-[hsl(var(--background))] shadow-sm',
      className
    )}>
      {children}
    </div>
  );
}

export function CardHeader({ children }: Props) {
  return <div className="px-6 py-4 border-b">{children}</div>;
}

export function CardBody({ children }: Props) {
  return <div className="px-6 py-4">{children}</div>;
}
```

6.2 StatCard with Memoization

StatCard displays a key metric with a label, value, and optional trend indicator. It is wrapped in React.memo to prevent unnecessary re-renders when parent components update but the stat values remain the same.

```
// StatCard.tsx
interface StatCardProps {
  label: string;
  value: string | number;
  trend?: 'up' | 'down' | 'neutral';
  trendValue?: string;
}

const StatCard = memo(function StatCard({
  label, value, trend, trendValue
}: StatCardProps) {
  return (
    <div className="relative overflow-hidden rounded-2xl ..." >
      { /* Animated gradient blob for visual interest */ }
      <div className="absolute -right-4 -top-4 h-24 w-24
        rounded-full bg-gradient-to-br from-[hsl(var(--primary)/0.2)]
        to-transparent will-change-transform" />
      <p className="text-sm text-muted">{label}</p>
      <p className="text-2xl font-bold">{value}</p>
      {trend && <TrendBadge dir={trend} val={trendValue} />}
    </div>
  );
});
```

will-change-transform for GPU Compositing

The gradient blob uses will-change-transform to hint to the browser that this element will be animated. This promotes it to its own GPU compositing layer, preventing repaints of the entire card when the blob animates. Use sparingly - overuse wastes GPU memory.

6.3 Badge Component

Badge renders a small colored label, commonly used for session outcomes, experiment statuses, and risk levels. It supports 6 variants through a variant prop.

Variant	Background	Text Color	Use Case
default	gray-100	gray-800	General labels
success	green-100	green-800	Converted, active
warning	amber-100	amber-800	Medium risk
destructive	red-100	red-800	High risk, failed
outline	transparent	gray-700	Subtle tags
primary	primary-100	primary-800	Active filters

6.4 Spinner, ErrorBox, and EmptyState

Three utility components handle loading, error, and empty states consistently across all pages.

Spinner

A CSS-animated spinning circle used as a loading indicator. It uses Tailwind's `animate-spin` utility class with a border-based spinner design.

ErrorBox with Retry

ErrorBox displays an error message with an optional retry button. The retry button calls the `refetch` function from `useQuery`, enabling users to recover from transient network errors without refreshing the page.

```
// ErrorBox.tsx
export function ErrorBox({
  message, onRetry
}: { message: string; onRetry?: () => void }) {
  return (
    <div className="rounded-xl border-destructive bg-red-50 p-4">
      <p className="text-destructive">{message}</p>
      {onRetry && (
        <button onClick={onRetry}
          className="mt-2 text-sm underline">Retry</button>
      )}
    </div>
  );
}
```

EmptyState

EmptyState renders a centered message when a query returns no results. It accepts a `message` prop and optionally an `icon`. Used in sessions list, experiments, and interventions pages.

Chapter 7

Overview Dashboard Page

7.1 Page Structure

The overview page (/dashboard) is the landing page that provides a high-level summary of the merchant's conversion performance. It consists of three sections: KPI cards at the top, a conversion funnel bar chart, and a risk distribution pie chart.

```
// dashboard/page.tsx
'use client';

import { useQuery } from '@lib/hooks';
import { getOverviewKPIs, getConversionFunnel,
  getRiskDistribution } from '@lib/api';
import { StatCard } from '@components/ui/StatCard';
import { LazyBarChart, LazyPieChart,
  LazyResponsiveContainer } from
  '@components/charts/LazyRecharts';

export default function OverviewPage() {
  const kpis = useQuery('overview-kpis', getOverviewKPIs);
  const funnel = useQuery('funnel', getConversionFunnel);
  const risk = useQuery('risk-dist', getRiskDistribution);
  // ... render
}
```

7.2 KPI StatCards

The top section renders 4 StatCard components in a responsive grid. Each card shows a key metric: total sessions, conversion rate, average risk score, and active interventions. The grid uses Tailwind's grid-cols system for responsive layout.

```
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6">
  <StatCard label="Total Sessions" value={kpis.data?.total_sessions}
    trend="up" trendValue="+12%" />
  <StatCard label="Conversion Rate" value={fmt.pct(kpis.data?.rate)}
    trend="up" trendValue="+3.2%" />
  <StatCard label="Avg Risk Score" value={fmt.dec(kpis.data?.risk)}
    trend="down" trendValue="-0.05" />
  <StatCard label="Active Interventions"
    value={kpis.data?.active_interventions} />
</div>
```

7.3 Lazy-Loaded Recharts

Charts are loaded dynamically using next/dynamic to avoid including the entire Recharts library (~400KB) in the initial JavaScript bundle. This is one of the most impactful performance optimizations in the application.

```
// components/charts/LazyRecharts.tsx
import dynamic from 'next/dynamic';

export const LazyBarChart = dynamic(
  () => import('recharts').then(m => m.BarChart),
  { ssr: false }
);
export const LazyPieChart = dynamic(
  () => import('recharts').then(m => m.PieChart),
  { ssr: false }
);
export const LazyLineChart = dynamic(
  () => import('recharts').then(m => m.LineChart),
  { ssr: false }
);
export const LazyResponsiveContainer = dynamic(
  () => import('recharts').then(m => m.ResponsiveContainer),
  { ssr: false }
);
```

Why ssr: false?

Recharts uses DOM APIs (SVG, getBBox) that are not available during server-side rendering. Setting `ssr: false` tells Next.js to only load and render these components on the client. The page first renders without charts, then they appear after the JS chunk loads.

7.4 Shared Chart Styles

To prevent re-renders caused by inline style objects, the chart tooltip and tick styles are defined as module-level constants. In React, an inline object like `style={{ fontSize: 12 }}` creates a new reference on every render, causing the Recharts component to re-render unnecessarily.

```
// Defined outside any component:
export const chartTooltipStyle = {
  borderRadius: '8px',
  border: '1px solid hsl(var(--border))',
  boxShadow: '0 4px 6px -1px rgba(0,0,0,0.1)',
};

export const tickStyle = {
  fontSize: 12,
  fill: 'hsl(var(--muted-foreground))',
};
```

7.5 Conversion Funnel Chart

The funnel chart uses LazyBarChart to display the conversion stages: `page_view`, `add_to_cart`, `checkout`, and `purchase`. Each bar represents the count at that stage, visually showing where users drop off.

7.6 Risk Distribution Pie Chart

The risk distribution pie chart uses LazyPieChart to show the breakdown of sessions by risk level (low, medium, high). Colors are mapped to the design system: green for low, amber for medium, red for high.

Chapter 8

Sessions List & Filtering (CONV-52)

8.1 Page Overview

The sessions page (/dashboard/sessions) displays a filterable, paginated table of all visitor sessions. It demonstrates several important React patterns: memoized filter components, stable options arrays, and dependency-tracked data fetching.

8.2 Filter State Management

All filter values are stored in a single state object. This keeps related state together and makes it easy to serialize for the useQuery dependency array.

```
interface Filters {
  outcome: string;
  device: string;
  risk: string;
  dateFrom: string;
  dateTo: string;
  page: number;
}

const [filters, setFilters] = useState<Filters>({
  outcome: '', device: '', risk: '',
  dateFrom: '', dateTo: '', page: 1,
});
```

8.3 Memoized Filter Components

The FilterSelect and FilterInput components are wrapped in React.memo to prevent re-renders when unrelated filters change. Without memo, changing the 'outcome' filter would re-render the 'device' filter component unnecessarily.

```
const FilterSelect = memo(function FilterSelect({
  label, value, onChange, options
}: FilterSelectProps) {
  return (
    <div>
      <label className="text-sm font-medium">{label}</label>
      <select value={value} onChange={e => onChange(e.target.value)}
        className="rounded-lg border px-3 py-2">
        <option value="">All</option>
        {options.map(o =>
          <option key={o} value={o}>{o}</option>)}
      </select>
    </div>
  );
});
```

8.4 Stable Options Arrays

The options for each filter dropdown are defined outside the component as module-level constants. This is a critical optimization - if the array were defined inside the component, React.memo would see a new reference on every render and re-render anyway.

```
// Defined OUTSIDE the component:
const OUTCOME_OPTIONS = ['converted', 'abandoned', 'bounced'];
const DEVICE_OPTIONS = ['desktop', 'mobile', 'tablet'];
const RISK_OPTIONS = ['low', 'medium', 'high'];

// Inside render:
<FilterSelect
  label="Outcome"
  value={filters.outcome}
  onChange={v => setFilters(f => ({ ...f, outcome: v, page: 1 })))}
  options={OUTCOME_OPTIONS} // stable reference!
/>
```

Gotcha: Filter Change Resets Page

Notice that changing any filter resets page to 1. Without this, the user could be on page 5, apply a filter that returns only 2 pages of results, and see an empty page. Always reset pagination when filters change.

8.5 Pagination

Pagination uses a memoized `PaginationBtn` component and calculates total pages from the API response. The page number is part of the filter state, so changing it triggers a `useQuery` refetch through the dependency array.

```
const PaginationBtn = memo(function PaginationBtn({
  page, current, onClick
}): { page: number; current: boolean; onClick: () => void } {
  return (
    <button
      onClick={onClick}
      className={clsx(
        'px-3 py-1 rounded-lg text-sm',
        current
          ? 'bg-[hsl(var(--primary))] text-white'
          : 'bg-[hsl(var(--secondary))]'
      )}
    >
      {page}
    </button>
  );
});
```

8.6 Session Table & Row Links

Each table row is a Next.js Link to the session detail page. The row displays session ID, visitor ID, outcome (as a Badge), risk score, device type, and created date. Clicking navigates to `/dashboard/sessions/[id]`.

Chapter 9

Session Detail View (CONV-53)

9.1 Page Structure

The session detail page (/dashboard/sessions/[id]) provides a deep dive into a single visitor session. It displays three sections: behavioral features, intervention recommendations, and an event timeline. The page uses the Next.js dynamic route parameter [id] to fetch session-specific data.

```
// sessions/[id]/page.tsx
'use client';

import { useParams } from 'next/navigation';

export default function SessionDetailPage() {
  const { id } = useParams<{ id: string }>();

  const session = useQuery(
    `session-${id}`,
    () => getSession(id)
  );
  const events = useQuery(
    `session-events-${id}`,
    () => getSessionEvents(id)
  );
  // ...
}
```

9.2 useCallback for Fetcher Functions

The fetcher functions are wrapped in useCallback to maintain stable references. While the fetcherRef in useQuery already handles stale closures, useCallback prevents unnecessary work in the hook itself.

```
const fetchSession = useCallback(
  () => getSession(id),
  [id]
);

const fetchEvents = useCallback(
  () => getSessionEvents(id),
  [id]
);

const session = useQuery(`session-${id}`, fetchSession);
const events = useQuery(`events-${id}`, fetchEvents);
```

9.3 Behavioral Features Grid

The features grid displays ML-extracted behavioral signals from the session. Each feature is shown in a `MetricBox` or `Feature` component with its name and value.

```
// Memoized feature display components
const MetricBox = memo(function MetricBox({
  label, value
}): { label: string; value: string | number }) {
  return (
    <div className="rounded-xl bg-[hsl(var(--secondary))] p-4">
      <p className="text-xs text-muted-foreground">{label}</p>
      <p className="text-lg font-semibold">{value}</p>
    </div>
  );
});

const Feature = memo(function Feature({
  name, value
}): { name: string; value: number }) {
  return (
    <div className="flex justify-between py-2 border-b">
      <span>{name}</span>
      <span className="font-mono">{value.toFixed(3)}</span>
    </div>
  );
});
```

9.4 Intervention Recommendations

Based on the session's risk score and behavioral features, the backend suggests intervention strategies. These are displayed as cards with the intervention type, confidence score, and expected impact.

9.5 Event Timeline Table

The bottom section shows a chronological table of all events in the session (page views, clicks, scrolls, cart actions,

etc.). Each row shows the event type, timestamp, and associated metadata.

```
<table className="w-full text-sm">
  <thead>
    <tr className="border-b text-muted-foreground">
      <th className="text-left py-2">Event</th>
      <th className="text-left py-2">Timestamp</th>
      <th className="text-left py-2">Details</th>
    </tr>
  </thead>
  <tbody>
    {events.data?.map((evt, i) => (
      <tr key={i} className="border-b">
        <td className="py-2">
          <Badge>{evt.event_type}</Badge>
        </td>
        <td>{formatTime(evt.timestamp)}</td>
        <td>{JSON.stringify(evt.metadata)}</td>
      </tr>
    ))}
  </tbody>
</table>
```

Cache Keys Include the Session ID

Notice that cache keys like 'session-abc123' and 'events-abc123' are unique per session. This means navigating between sessions creates separate cache entries, allowing instant back-navigation to a previously viewed session within the 30s TTL window.

Chapter 10

Analytics Dashboard (CONV-54)

10.1 Page Layout

The analytics page (/dashboard/analytics) is the most chart-heavy page in the application. It contains four visualization sections: conversion funnel, friction hotspots, risk distribution, and cohort analysis. Each section fetches its data independently via useQuery.

```
// analytics/page.tsx
'use client';

export default function AnalyticsPage() {
  const funnel = useQuery('analytics-funnel', getConversionFunnel);
  const friction = useQuery('analytics-friction', getFrictionPoints);
  const riskDist = useQuery('analytics-risk', getRiskDistribution);
  const [cohortDim, setCohortDim] = useState('device');
  const cohorts = useQuery(
    `analytics-cohorts-${cohortDim}`,
    () => getCohortAnalysis(cohortDim),
    [cohortDim]
  );
  // ...
}
```

10.2 Conversion Funnel (Bar Chart)

The funnel visualization uses LazyBarChart with horizontal bars showing progression through conversion stages. Each bar represents a stage count, with color intensity decreasing to visually convey the funnel narrowing.

10.3 Friction Hotspot Table

The friction analysis identifies pages and interactions where users experience the most difficulty. Data is displayed in a table sorted by friction score, with each row showing the page URL, average time on page, rage click count, and drop-off rate.

```
// Friction hotspots table
<table className="w-full text-sm">
  <thead>
    <tr className="text-muted-foreground border-b">
      <th className="text-left py-2">Page</th>
      <th className="text-right py-2">Friction Score</th>
      <th className="text-right py-2">Avg Time (s)</th>
      <th className="text-right py-2">Drop-off %</th>
    </tr>
  </thead>
  <tbody>
    {friction.data?.map((row, i) => (
      <tr key={i} className="border-b">
        <td className="py-2">{row.page}</td>
        <td className="text-right">{row.score.toFixed(2)}</td>
        <td className="text-right">{row.avg_time}</td>
        <td className="text-right">{fmt.pct(row.dropoff)}</td>
      </tr>
    ))}
  </tbody>
</table>
```

10.4 Risk Distribution (Bar Chart)

Unlike the overview's pie chart, the analytics page uses a bar chart for risk distribution to show more precise counts. Bars are color-coded: green for low risk, amber for medium, and red for high.

10.5 Cohort Analysis with Dimension Selector

The cohort analysis section is the most interactive chart. Users can select a dimension (device, browser, country, traffic source) to group sessions and compare conversion rates across cohorts.

```
// Cohort dimension selector
const COHORT_DIMENSIONS = ['device', 'browser', 'country', 'source'];

<select
  value={cohortDim}
  onChange={e => setCohortDim(e.target.value)}
  className="rounded-lg border px-3 py-2"
>
  {COHORT_DIMENSIONS.map(d =>
    <option key={d} value={d}>{d}</option>)}
</select>
```

Changing the dimension updates the cohortDim state, which changes the useQuery cache key (analytics-cohorts-device vs analytics-cohorts-browser), triggering a new API call. Previously fetched dimensions are cached for 30 seconds.

10.6 Memoized Tooltip Formatters

Recharts accepts formatter functions for tooltips and axis labels. These are wrapped in useMemo to prevent re-renders:

```
const tooltipFormatter = useMemo(
  () => (value: number) => [fmt.number(value), 'Count'],
  []
);

const pctFormatter = useMemo(
  () => (value: number) => [fmt.pct(value), 'Rate'],
  []
);

// Used in Recharts:
<Tooltip formatter={tooltipFormatter}
  contentStyle={chartTooltipStyle} />
```

Combo Charts: Line + Bar

The cohort analysis uses a composed chart (ComposedChart from Recharts) that combines bars for session count and a line for conversion rate. This dual-axis visualization lets merchants compare volume and performance in a single view.

Chapter 11

Experiments Management (CONV-55)

11.1 Page Architecture

The experiments page (/dashboard/experiments) provides full CRUD (Create, Read, Update, Delete) capabilities for A/B experiments. The page is composed of three main sub-components: a create form, experiment cards, and a stats panel.

```
// experiments/page.tsx structure
'use client';

export default function ExperimentsPage() {
  const [showCreate, setShowCreate] = useState(false);
  const [selectedId, setSelectedId] = useState<string | null>(null);
  const experiments = useQuery('experiments', getExperiments);

  return (
    <div>
      <header className="flex justify-between items-center mb-6">
        <h1>Experiments</h1>
        <button onClick={() => setShowCreate(true)}>
          + New Experiment
        </button>
      </header>

      {showCreate && <CreateForm onClose={() => {
        setShowCreate(false);
        experiments.refetch();
      }} />}

      <div className="grid gap-4">
        {experiments.data?.map(exp =>
          <ExperimentCard key={exp.id} experiment={exp}
            onSelect={() => setSelectedId(exp.id)} />
        )}
      </div>

      {selectedId && <StatsPanel id={selectedId}
        onClose={() => setSelectedId(null)} />}
    </div>
  );
}
```

11.2 CreateForm Component

The CreateForm collects experiment details: name, description, hypothesis, variant names, and traffic allocation

percentage. On submit, it calls the `createExperiment()` API function and triggers a refetch of the experiments list.

```
// CreateForm submit handler
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setSubmitting(true);
  try {
    await createExperiment({
      name: form.name,
      description: form.description,
      hypothesis: form.hypothesis,
      variants: form.variants.split(',').map(v => v.trim()),
      traffic_pct: Number(form.traffic_pct),
    });
    onClose(); // triggers refetch in parent
  } catch (err) {
    setError(err instanceof ApiError ? err.detail : 'Failed');
  } finally {
    setSubmitting(false);
  }
};
```

11.3 ExperimentCard Component

Each experiment is displayed in a card showing its name, status (as a Badge), creation date, and a button to view stats. The card uses conditional styling based on experiment status (active, paused, completed).

11.4 StatsPanel Component

When a user clicks 'View Stats' on an experiment card, the StatsPanel slides in showing variant-level performance metrics. It fetches data via `getExperimentStats(id)`.

```
function StatsPanel({ id, onClose }: StatsPanelProps) {
  const stats = useQuery(
    `experiment-stats-${id}`,
    () => getExperimentStats(id)
  );

  return (
    <Card>
      <CardHeader>
        <h3>Experiment Results</h3>
        <button onClick={onClose}>Close</button>
      </CardHeader>
      <CardBody>
        {stats.data?.variants.map(v => (
          <div key={v.name} className="flex justify-between">
            <span>{v.name}</span>
            <span>Conv: {fmt.pct(v.conversion_rate)}</span>
            <span>n={v.sample_size}</span>
          </div>
        ))}
        {stats.data?.winner &&
          <Badge variant="success">
            Winner: {stats.data.winner}
          </Badge>
        }
      </CardBody>
    </Card>
  );
}
```

Refetch Pattern After Mutation

After creating an experiment, we call `onClose()` which triggers the parent to call `experiments.refetch()`. This ensures the list shows the new experiment immediately. An alternative is optimistic updates (adding to local state before API confirms), but refetch is simpler and guarantees consistency with the server.

Chapter 12

Interventions Dashboard (CONV-55)

12.1 Page Layout

The interventions page (/dashboard/interventions) visualizes the effectiveness of automated interventions (popups, discounts, urgency messages, etc.) that the system triggers based on ML risk predictions. The page has two sections: a stacked bar chart and a detailed statistics table.

12.2 Stacked Bar Chart

The main visualization is a stacked bar chart where each bar represents an intervention type, and the segments show the outcome breakdown: converted (green), dismissed (gray), ignored (amber), and bounced (red).

```
// Stacked bar chart for interventions
<LazyResponsiveContainer width="100%" height={400}>
  <LazyBarChart data={interventions.data} layout="vertical">
    <XAxis type="number" />
    <YAxis type="category" dataKey="type" width={120} />
    <Tooltip
      formatter={tooltipFormatter}
      contentStyle={chartTooltipStyle}
    />
    <Legend />
    <Bar dataKey="converted" stackId="a" fill="#22c55e" />
    <Bar dataKey="dismissed" stackId="a" fill="#94a3b8" />
    <Bar dataKey="ignored" stackId="a" fill="#f59e0b" />
    <Bar dataKey="bounced" stackId="a" fill="#ef4444" />
  </LazyBarChart>
</LazyResponsiveContainer>
```

stackId Groups Bars Together

In Recharts, Bar components with the same `stackId` are stacked on top of each other instead of placed side by side. This creates a single bar per intervention type showing the proportional outcomes. Remove `stackId` to get a grouped bar chart instead.

12.3 Tooltip Formatter

The tooltip formatter is memoized to prevent re-renders and formats the count value with thousand separators.

```
const tooltipFormatter = useMemo(
  () => (value: number, name: string) => [
    fmt.number(value),
    name.charAt(0).toUpperCase() + name.slice(1),
  ],
  []
);
```

12.4 Intervention Stats Table

Below the chart, a detailed table shows per-intervention metrics: total triggers, conversion rate, average time to convert, dismiss rate, and ROI estimate. This data comes from the `getInterventionStats()` endpoint.

```
// Stats table rendering
<table className="w-full text-sm">
  <thead>
    <tr className="border-b">
      <th className="text-left py-2">Intervention</th>
      <th className="text-right">Triggers</th>
      <th className="text-right">Conv Rate</th>
      <th className="text-right">Avg Time</th>
      <th className="text-right">Dismiss %</th>
    </tr>
  </thead>
  <tbody>
    {stats.data?.map((s, i) => (
      <tr key={i} className="border-b">
        <td className="py-2">{s.type}</td>
        <td className="text-right">{fmt.number(s.triggers)}</td>
        <td className="text-right">{fmt.pct(s.conv_rate)}</td>
        <td className="text-right">{s.avg_time_s}s</td>
        <td className="text-right">{fmt.pct(s.dismiss_rate)}</td>
      </tr>
    ))}
  </tbody>
</table>
```

Chapter 13

Settings & SDK Key Management

13.1 Page Overview

The settings page (/dashboard/settings) manages the merchant's SDK key and profile. It is the only page that writes to localStorage (all other pages only read from it). The page demonstrates client-side state management without full-page reloads.

13.2 SDK Key Management

The SDK key is the authentication token used in all API requests. The settings page provides three operations: view the current key, save a new key, and rotate (regenerate) the key.

```
// Settings page SDK key management
const [sdkKey, setSdkKey] = useState('');
const [saved, setSaved] = useState(false);

useEffect(() => {
  const key = localStorage.getItem('sdkKey') ?? '';
  setSdkKey(key);
}, []);

const handleSave = () => {
  localStorage.setItem('sdkKey', sdkKey);
  setSaved(true);
  setTimeout(() => setSaved(false), 2000);
};
```

Client-Side Save Feedback (No Reload!)

An earlier version used `window.location.reload()` after saving the SDK key. This was replaced with a 'saved' state flag that shows a green checkmark for 2 seconds. This is much better UX - the user gets instant feedback without losing page state.

13.3 Key Rotation

Key rotation calls a backend endpoint to generate a new SDK key. The new key is automatically saved to localStorage and the input field is updated.

```
const handleRotate = async () => {
  if (!confirm('Rotate SDK key? The old key will stop working.')) {
    return;
  }
  try {
    const result = await rotateSdkKey();
    setSdkKey(result.sdk_key);
    localStorage.setItem('sdkKey', result.sdk_key);
    setSaved(true);
    setTimeout(() => setSaved(false), 2000);
  } catch (err) {
    setError('Failed to rotate key');
  }
};
```

Security: Confirm Before Rotation

Key rotation invalidates the old key immediately. All active SDK integrations will break until updated with the new key. The confirm() dialog prevents accidental rotation. In production, consider a grace period where both keys work.

13.4 Merchant Profile Display

The bottom section of the settings page displays the merchant profile fetched from the API. This includes the merchant name, plan, created date, and contact email. It uses the getMerchant() endpoint.

```
const merchant = useQuery('merchant', getMerchant);

<Card>
  <CardHeader>Merchant Profile</CardHeader>
  <CardBody>
    <div className="grid grid-cols-2 gap-4">
      <div>
        <p className="text-sm text-muted">Name</p>
        <p className="font-medium">{merchant.data?.name}</p>
      </div>
      <div>
        <p className="text-sm text-muted">Plan</p>
        <Badge variant="primary">{merchant.data?.plan}</Badge>
      </div>
    </div>
  </CardBody>
</Card>
```


Chapter 14

Formatting Utilities

14.1 Overview

The `format.ts` module (`src/lib/format.ts`) provides 7 pure utility functions for consistent data formatting across the application. All functions are stateless and side-effect-free, making them easy to test and reuse.

14.2 Function Reference

1. `fmt.number(value: number): string`

Formats a number with thousand separators using `Intl.NumberFormat`. Handles undefined and NaN gracefully by returning '0'.

```
export function number(value: number | undefined): string {
  if (value == null || isNaN(value)) return '0';
  return new Intl.NumberFormat('en-US').format(value);
}

// Examples:
// number(1234567) => '1,234,567'
// number(42)      => '42'
// number(undefined) => '0'
```

2. `fmt.pct(value: number): string`

Formats a decimal value (0-1) as a percentage string with one decimal place.

```
export function pct(value: number | undefined): string {
  if (value == null || isNaN(value)) return '0.0%';
  return (value * 100).toFixed(1) + '%';
}

// Examples:
// pct(0.856) => '85.6%'
// pct(0.1)   => '10.0%'
// pct(1)     => '100.0%'
```

3. `fmt.dec(value: number, places?: number): string`

Formats a number with a specified number of decimal places (default 2).

```
export function dec(
  value: number | undefined, places = 2
): string {
  if (value == null || isNaN(value)) return '0.00';
  return value.toFixed(places);
}

// Examples:
// dec(3.14159)    => '3.14'
// dec(3.14159, 4) => '3.1416'
```

4. **fmt.date(value: string): string**

Formats an ISO date string to a human-readable date using the browser locale.

```
export function date(value: string | undefined): string {
  if (!value) return '-';
  return new Date(value).toLocaleDateString('en-US', {
    year: 'numeric', month: 'short', day: 'numeric',
  });
}

// Examples:
// date('2026-03-21T10:30:00Z') => 'Mar 21, 2026'
```

5. **fmt.time(value: string): string**

Formats an ISO date string to a time string with hours, minutes, and seconds.

```
export function time(value: string | undefined): string {
  if (!value) return '-';
  return new Date(value).toLocaleTimeString('en-US', {
    hour: '2-digit', minute: '2-digit', second: '2-digit',
  });
}
```

6. **fmt.duration(seconds: number): string**

Converts a number of seconds into a human-readable duration string (e.g., '2m 30s').

```
export function duration(seconds: number | undefined): string {
  if (seconds == null || isNaN(seconds)) return '0s';
  const m = Math.floor(seconds / 60);
  const s = Math.round(seconds % 60);
  return m > 0 ? `${m}m ${s}s` : `${s}s`;
}

// Examples:
// duration(150) => '2m 30s'
// duration(45)  => '45s'
```

7. **fmt.truncate(text: string, max?: number): string**

Truncates a string to a maximum length with an ellipsis.

```
export function truncate(
  text: string, max = 50
): string {
  if (!text || text.length <= max) return text ?? '';
  return text.slice(0, max) + '...';
}
```

Why Intl.NumberFormat?

Using Intl.NumberFormat for number formatting respects the user's locale settings and handles edge cases (negative numbers, very large numbers) correctly. It is built into all modern browsers with zero bundle cost, unlike libraries like numeral.js.

Chapter 15

Performance Optimization Deep Dive

15.1 Bundle Size: Lazy Imports

The single largest optimization is lazy-loading Recharts via next/dynamic. Recharts and its D3 dependencies account for approximately 400KB of JavaScript. By using dynamic imports, this code is split into a separate chunk that only loads when a chart component first renders.

```
// Without lazy loading:
import { BarChart } from 'recharts'; // 400KB in main bundle

// With lazy loading:
const LazyBarChart = dynamic(
  () => import('recharts').then(m => m.BarChart),
  { ssr: false }
);
// 0KB in main bundle, 400KB loaded on demand
```

Combined with next.config.js optimizePackageImports for recharts, this ensures that only the specific chart components used are included in the dynamic chunk, further reducing the loaded size.

15.2 React.memo Strategy

React.memo is applied to components that meet two criteria: (1) they render frequently due to parent re-renders, and (2) their props change infrequently. Memoized components in this project include:

Component	Why Memoized
StatCard	Parent (Overview) re-renders on any KPI change; individual cards rarely change
FilterSelect	Other filter changes cause parent re-render; this filter's props are stable
FilterInput	Same as FilterSelect for text input fields
PaginationBtn	Only current page button changes; others should skip re-render
MetricBox	Session detail features grid re-renders; individual boxes are static
Feature	Same as MetricBox for feature list items

When NOT to Use React.memo

React.memo has overhead: it must shallow-compare all props on every render. If a component's props change on every render (e.g., inline objects, new function references), memo actually makes performance worse. Only memoize after verifying props are stable.

15.3 Stable References Pattern

Several patterns in the codebase ensure that object and function references remain stable across renders:

1. Module-level constants: `OUTCOME_OPTIONS`, `DEVICE_OPTIONS`, `chartTooltipStyle`, `tickStyle` are defined outside components so they are created once.
2. `useMemo` for computed values: `tooltipFormatter` functions are memoized with empty dependency arrays.
3. `useCallback` for event handlers: Filter change handlers and fetcher functions use `useCallback` to maintain stable references.
4. `fetcherRef` in `useQuery`: The `ref` pattern avoids putting the fetcher function in the effect dependency array.

15.4 Cache Deduplication

The `useQuery` cache prevents duplicate API calls when multiple components request the same data within 30 seconds. For example, navigating away from the overview page and back within 30 seconds returns cached data instantly without an API call.

The cache also deduplicates concurrent requests. If two components mount simultaneously with the same cache key, only one API call is made (the second finds cached data).

15.5 Next.js Configuration Optimizations

```
// next.config.js
const nextConfig = {
  reactStrictMode: true,    // Catches bugs in development
  compress: true,          // gzip compression for responses
  images: {
    formats: ['image/avif', 'image/webp'], // Modern formats
  },
  experimental: {
    optimizePackageImports: ['recharts'], // Tree-shake recharts
  },
};
```

1. `reactStrictMode`: Runs effects twice in development to catch side effects and stale closures. No impact in production.
2. `compress`: Enables gzip compression for all HTTP responses, reducing transfer size by ~70%.
3. `image formats`: Serves AVIF (best compression) with WebP fallback. AVIF saves ~50% vs JPEG.
4. `optimizePackageImports`: Enables deep tree-shaking for `recharts`, eliminating unused chart types from the bundle.

15.6 GPU Compositing for Animations

The `StatCard` gradient blob uses `will-change: transform` (via Tailwind's `will-change-transform`) to promote the element to its own GPU compositing layer. This means the blob animation runs on the GPU without triggering layout recalculations or repaints of surrounding elements.

15.7 LCP Optimization

The Sidebar logo uses Next.js Image with the priority prop, which adds a `<link rel='preload'>` tag in the document head. Since the logo is the largest contentful paint (LCP) element on initial load, preloading it ensures it appears as quickly as possible.

Chapter 16

Build, Deploy & Next Steps

16.1 Build Configuration

The production build process uses Next.js's built-in optimization pipeline. Running 'npm run build' produces an optimized production bundle with automatic code splitting, tree shaking, and minification.

```
# Development
npm run dev      # Starts dev server on localhost:3000

# Production build
npm run build    # Compiles and optimizes
npm run start    # Serves production build

# Type checking
npx tsc --noEmit # Verify types without emitting files
```

16.2 Environment Variables

The application uses a single environment variable for configuration. Next.js requires the NEXT_PUBLIC_ prefix for variables that must be available in the browser.

Variable	Default	Description
NEXT_PUBLIC_API_URL	http://localhost:8000	Backend API base URL

NEXT_PUBLIC_ Prefix is Required

Without the NEXT_PUBLIC_ prefix, environment variables are only available on the server (in API routes and Server Components). Since our API client runs in the browser, the variable **MUST** have the NEXT_PUBLIC_ prefix to be included in the client bundle.

16.3 CORS Configuration

The backend was updated to allow cross-origin requests from the frontend development servers. The CORS configuration in the backend config.py includes both localhost:3000 (default Next.js port) and localhost:3001 (alternate port).

```
# Backend config.py CORS settings
CORS_ORIGINS = os.getenv('CORS_ORIGINS',
    'http://localhost:3000,http://localhost:3001'
).split(',')

# Backend .env override
CORS_ORIGINS=http://localhost:3000,http://your-domain.com
```

16.4 Production Checklist

Before deploying to production, verify the following items:

1. Set NEXT_PUBLIC_API_URL to the production backend URL (e.g., <https://api.conversiono.com>).
2. Update CORS_ORIGINS in the backend to include the production frontend domain.
3. Replace localStorage SDK key storage with httpOnly cookies or a secure token exchange.
4. Enable CSP (Content Security Policy) headers to prevent XSS attacks.
5. Configure a CDN (CloudFront, Vercel Edge) for static assets and the Next.js build output.
6. Set up monitoring (Sentry, Datadog) for runtime error tracking.
7. Run Lighthouse audit to verify Core Web Vitals (LCP < 2.5s, FID < 100ms, CLS < 0.1).
8. Enable rate limiting on the backend to prevent API abuse.
9. Add E2E tests (Playwright or Cypress) for critical user flows.
10. Review bundle analysis output (npm run build shows chunk sizes) for unexpected large modules.

16.5 Next Steps - Epic 6 and Beyond

The Merchant Dashboard v1 establishes the foundation. Future enhancements planned for subsequent epics include:

- Real-time updates via WebSocket for live session monitoring and KPI streaming.
- Dark mode support using the CSS custom property design system (swap :root token values).
- Advanced filtering with saved filter presets and URL-based filter state for shareable links.
- Export capabilities: CSV/PDF export for analytics data and experiment reports.
- Role-based access control (RBAC) with different permission levels for merchant team members.
- Onboarding wizard to guide new merchants through SDK integration and dashboard setup.
- A/B test statistical significance calculator with Bayesian inference visualization.
- Custom dashboard layouts allowing merchants to arrange widgets and pin favorite metrics.

Architecture Ready for Growth

The layered architecture, typed API client, and caching hook were designed to support these future features without major refactoring. Adding WebSocket support, for example, only requires a new hook that writes to the same cache, and all existing components will automatically display the live data.